What Is System Design?

HLD vs LLD, and what interviewers look for

Watch the 2-min video

In one line: system design is turning requirements into an architecture: components, the data flows between them, and the trade-offs you choose to accept.

1 From requirements to an architecture

Every design starts from two kinds of requirements. **Functional** requirements say what the system must do: a user can shorten a URL, a driver can accept a ride. **Non-functional** requirements say how well it must do it: answer in under 200 ms, stay up 99.9% of the time, survive a data-centre outage, cost less than a set budget.

Design is the step from that list to a picture you can build: which **components** exist (services, databases, caches, queues), how **data flows** between them (who calls whom, synchronously or through a queue), and which **trade-offs** you accept to meet the numbers. Two engineers given the same requirements can draw different, equally valid designs; what makes one better is how well it fits the constraints, and how clearly its trade-offs are explained.

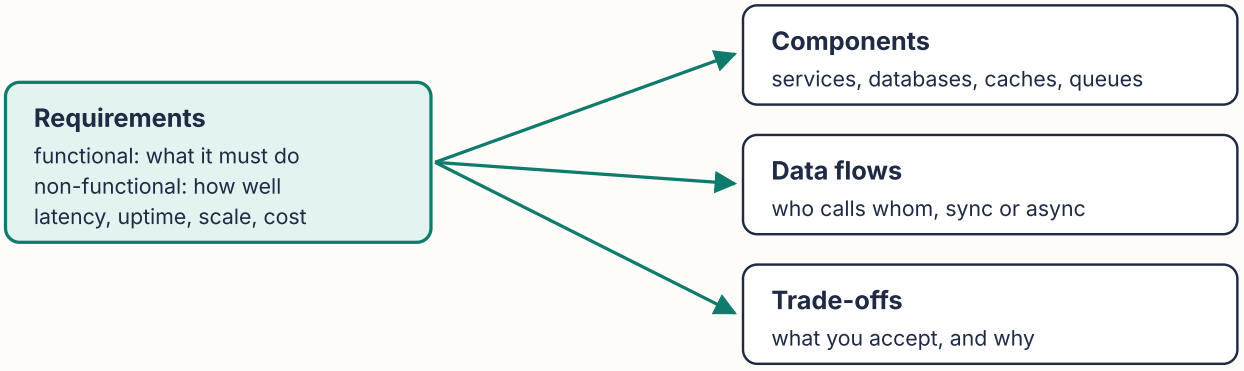


Figure 1. A design answers three questions about the requirements: what the parts are, how they talk, and what you give up.

2 Two levels: high-level and low-level design

High-level design (HLD) is the map of the system: services, data stores, networks, and how requests travel through them. It answers questions like “where is data stored?”, “what happens when this service fails?” and “how does it scale to 10× traffic?”.

Low-level design (LLD) zooms into one box of that map and designs its insides: classes, modules, interfaces, method signatures, and design patterns. It answers “how is this code organised so it is easy to test and change?”.

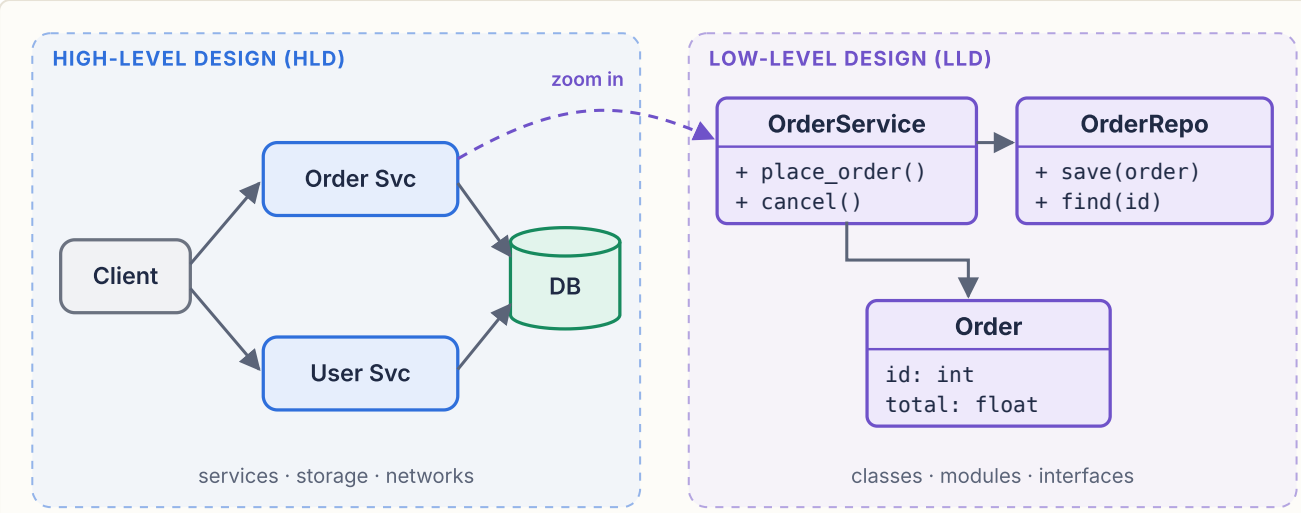


Figure 2. The same order service seen at two zoom levels: as one box in the system (left), and as classes inside that box (right).

	High-level design (HLD)	Low-level design (LLD)
Scope	The whole system	One service or module
Building blocks	Services, databases, caches, queues, load balancers, CDNs	Classes, interfaces, functions, design patterns
Typical questions	Scale, availability, consistency, latency, cost	Responsibilities, extensibility, testability
Output	Architecture diagram, data flow, capacity estimates	Class diagram, API contracts, schema details
Interview round	“Design Twitter / a URL shortener”	“Design a parking lot / an LRU cache”

3 Worked example: a FastAPI app grows up

Designs are rarely drawn complete on day one. They grow as load grows, one bottleneck at a time. Follow a small FastAPI app as its traffic rises.

1. **One server.** The app and its database share one machine. Cheap and simple, but the two compete for CPU, memory and disk, and one crash takes everything down.
2. **Separate the database.** The database moves to its own machine, sized for storage and memory. The app can now be restarted or redeployed without touching data.
3. **Scale the app tier.** A load balancer spreads requests over several identical copies of the app. One copy can fail without an outage, and you add copies as traffic grows. This works only if the app is *stateless* (no user session kept in a server's memory).
4. **Cache and CDN.** A cache such as Redis answers repeated reads in microseconds and protects the database. A CDN serves images, scripts and video from locations close to users.

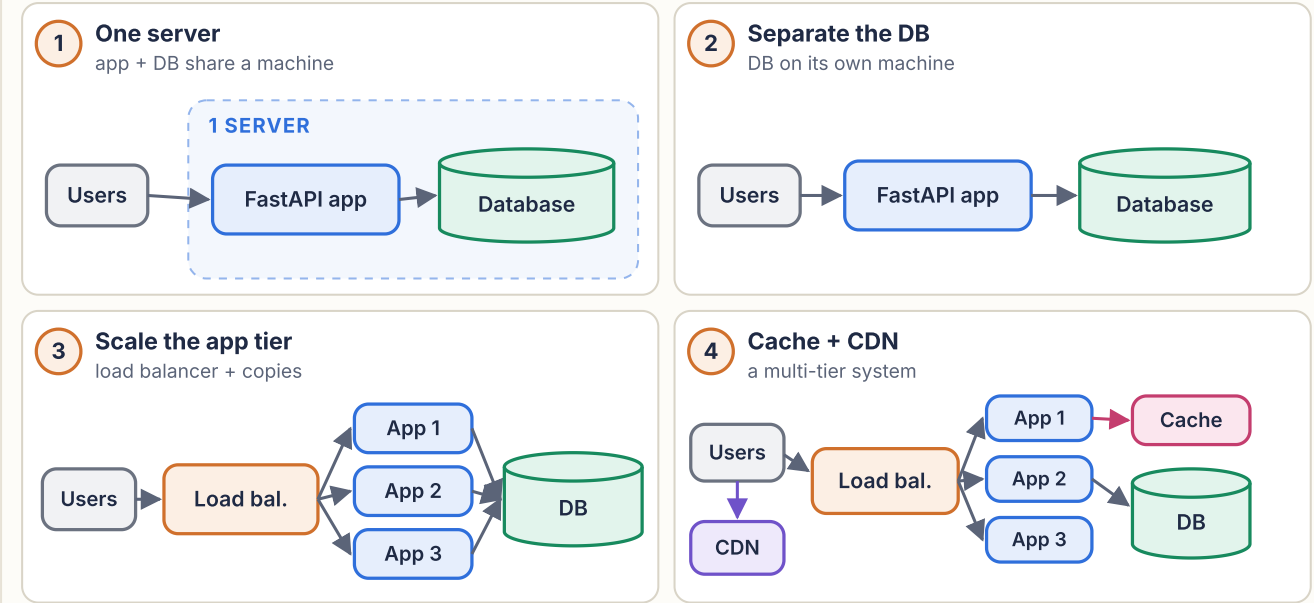


Figure 3. Four steps from one server to a multi-tier system. Each step fixes the bottleneck the previous one hit; in step 4, hot reads go to the cache and everything else to the database.

4 Every step is a trade-off

Step	What you gain	What it costs
Separate database	Independent scaling, safer deploys	A network hop on every query, a second machine to run
Load balancer + copies	Higher availability, more throughput	More servers, stateless code required, more to monitor
Cache	Much lower read latency, less database load	Stale data risk, invalidation logic, another system to fail
CDN	Fast static content worldwide	Cost per GB, cache purging on every release

HOW TO TALK ABOUT TRADE-OFFS

Name the option, the alternative, and the constraint that decides it: “I'll add a cache because reads outnumber writes 100 to 1 and the product page can tolerate data that is a few seconds old. If prices had to be exact, I would read prices from the database and cache only descriptions.”

5 What interviewers actually evaluate

A system design interview has no single right answer. The interviewer scores how you get to a design:

- **Structured thinking.** Clarify requirements and scale before drawing a box. Move from the big picture to the details.
- **Trade-offs.** For each choice, say why it beats the alternative here, and what would change your mind.
- **Communication.** Think out loud, check in with the interviewer, and adapt when they push on a part.

A TYPICAL 45-MINUTE ROUND

~5 min requirements and scope · ~5 min capacity estimates · ~10 min high-level design · ~15 min deep dives on 1–2 parts the interviewer picks · ~5 min bottlenecks, failures and wrap-up. Phase 8 of this course practises this exact structure.

COMMON MISTAKES

- Drawing boxes before asking what the system must do and at what scale.
- Naming technologies (Kafka, Cassandra) without saying why they fit.
- Designing for Google scale when the requirements say 1,000 users.
- Going silent for minutes; the interviewer can only score what you say.

6 Key terms

Functional requirement: what the system must do

Non-functional requirement: how well: latency, availability, cost

HLD: system-level architecture

LLD: component-level code design

Load balancer: spreads requests over servers

Stateless service: keeps no user state in memory, so any copy can serve any request

Cache: fast copy of hot data

CDN: servers near users for static content

7 Interview questions

Q1. What does a system design interview evaluate? BASIC

How you think, not one correct answer: whether you clarify requirements, structure the design from the big picture down, justify trade-offs against the constraints, and communicate while you work.

Q2. How is high-level design different from low-level design? BASIC

HLD covers the whole system: services, storage, networks and data flow, judged on scale, availability and cost. LLD covers the inside of one component: classes, interfaces and patterns, judged on clarity, extensibility and testability.

Q3. Give functional and non-functional requirements for a URL shortener. BASIC

Functional: create a short link, redirect to the long URL, optional custom alias and expiry. Non-functional: redirects in under ~100 ms at p99, very high availability for redirects (they are most of the traffic), hundreds of millions of links, and short codes that never collide.

Q4. The interviewer says "Design Instagram." What do you do in the first five minutes? SCENARIO

Clarify scope (photo upload, follow, feed; leave out DMs and stories unless asked), scale (daily users, uploads per day, read/write ratio), and non-functional goals (feed latency, availability, how fresh the feed must be). Confirm the list with the interviewer before drawing anything.

Q5. Your single-server app is slowing down as traffic grows. What do you do first? SCENARIO

Measure where the time goes. Usually: move the database to its own machine, then put a load balancer in front of several stateless app copies, then add a cache for hot reads. Each step targets the current bottleneck, not a guess.

Q6. You add two more app servers behind a load balancer. What must change in the code? SCENARIO

The app must become stateless: sessions move to Redis or signed tokens, uploaded files go to object storage instead of local disk, and in-memory caches can no longer be assumed shared. The load balancer also needs health checks so it stops sending traffic to a broken copy.

Q7. Why is “add a cache” not a free improvement? TRICKY

It trades freshness for speed: cached data can be stale, invalidation adds complexity, and the cache is one more component that can fail or fill up. It is worth it when reads dominate and slightly old data is acceptable.

Q8. If there's no single right answer, how do you know a design is good? TRICKY

It meets the stated requirements with numbers, has no unexplained single point of failure, every major choice comes with its reason and the alternative, and you can say what breaks first at 10× the load and how you'd fix it.

Next: SD-A-002 · Client-Server & Multi-Tier Architecture backendroadmap.com